JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface BiPredicate<T,U>

Type Parameters:

T - the type of the first argument to the predicate

U - the type of the second argument the predicate

Functional Interface:

This is a functional interface and can therefore be used as the assignment target for a lambda expression or method reference.

```
@FunctionalInterface
public interface BiPredicate<T,U>
```

Represents a predicate (boolean-valued function) of two arguments. This is the two-arity specialization of `Predicate`.

This is a functional interface whose functional method is `test(Object, Object)`.

Since:

1.8

See Also:

`Predicate`

Method Summary

All Methods	Instance Methods	
Modifier and Type	**Method**	**Description**
`default BiPredicate<T,U>`	`and(BiPredicate<? super T, ? super U> other)`	Returns a composed predicate that represents a short-circuiting logical AND of this predicate and another.
`default BiPredicate<T,U>`	`negate()`	Returns a predicate that represents the logical negation of this predicate.
`default BiPredicate<T,U>`	`or(BiPredicate<? super T, ? super U> other)`	Returns a composed predicate that represents a short-circuiting logical OR of

		this predicate and another.
boolean	test(T t, U u)	Evaluates this predicate on the given arguments.

Method Details

test

```
boolean test(T t,
             U u)
```

Evaluates this predicate on the given arguments.

Parameters:

`t` - the first input argument

`u` - the second input argument

Returns:

`true` if the input arguments match the predicate, otherwise `false`

and

```
default BiPredicate<T,U> and(BiPredicate<? super T, ? super U> other)
```

Returns a composed predicate that represents a short-circuiting logical AND of this predicate and another. When evaluating the composed predicate, if this predicate is `false`, then the `other` predicate is not evaluated.

Any exceptions thrown during evaluation of either predicate are relayed to the caller; if evaluation of this predicate throws an exception, the `other` predicate will not be evaluated.

Parameters:

`other` - a predicate that will be logically-ANDed with this predicate

Returns:

a composed predicate that represents the short-circuiting logical AND of this predicate and the `other` predicate

Throws:

`NullPointerException` - if other is null

negate

```
default BiPredicate<T,U> negate()
```

Returns a predicate that represents the logical negation of this predicate.

Returns:

a predicate that represents the logical negation of this predicate

or

```
default BiPredicate<T,U> or(BiPredicate<? super T, ? super U> other)
```

Returns a composed predicate that represents a short-circuiting logical OR of this predicate and another. When evaluating the composed predicate, if this predicate is `true`, then the `other` predicate is not evaluated.

Any exceptions thrown during evaluation of either predicate are relayed to the caller; if evaluation of this predicate throws an exception, the `other` predicate will not be evaluated.

Parameters:

`other` - a predicate that will be logically-ORed with this predicate

Returns:

a composed predicate that represents the short-circuiting logical OR of this predicate and the `other` predicate

Throws:

`NullPointerException` - if other is null

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.